

Why We Should Trust Binary NDCG@3 More

llm-knowledge-enhancement

1 Intro

For a user, let t be the item they actually bought next, and let p_1, p_2, p_3 be the top 3 items our system recommends, in order.

Every item has an embedding vector, and those vectors are unit-normalized (length 1). So for any two items a and b , the dot product $v_a \cdot v_b$ is just their cosine similarity: 1 if they point the same way, 0 if unrelated, and it can dip negative for opposite items.

Old metric: NDCG@3 using embedding similarity

$$\text{rel}_i = \max(0, v_t \cdot v_{p_i}) \quad (1)$$

$$\text{DCG} = \sum_{i=1}^3 \frac{\text{rel}_i}{\log_2(i+1)} \quad (2)$$

$$\text{IDCG} = \text{DCG computed after sorting the rel values from biggest to smallest} \quad (3)$$

$$\text{NDCG@3} = \frac{\text{DCG}}{\text{IDCG}} \quad (4)$$

So instead of asking “did we recommend t ”, this asks “how similar, on average, were our 3 guesses to t , weighted by rank”. Getting the dot product with t close to 1 counts almost as much as getting t exactly right.

New metric: NDCG@3, binary

Only one item counts as relevant: t itself. So the best possible score is always 1 (put t at rank 1), which makes IDCG always equal to 1 and the formula collapses to just:

$$\text{NDCG@3}_{\text{binary}} = \begin{cases} \frac{1}{\log_2(r+1)} & \text{if } t \text{ shows up at rank } r \leq 3 \\ 0 & \text{if } t \text{ never shows up in the top 3} \end{cases} \quad (5)$$

That’s it. No dot products, no normalizing step. Either we found the exact item or we didn’t, and how early we found it decides the score.

1.1 Metric output

If $\text{NDCG@3}_{\text{binary}} = f(r)$, where r is the ranked item ordinality, we would have an output of:

$$0 < f(r) \leq 1$$

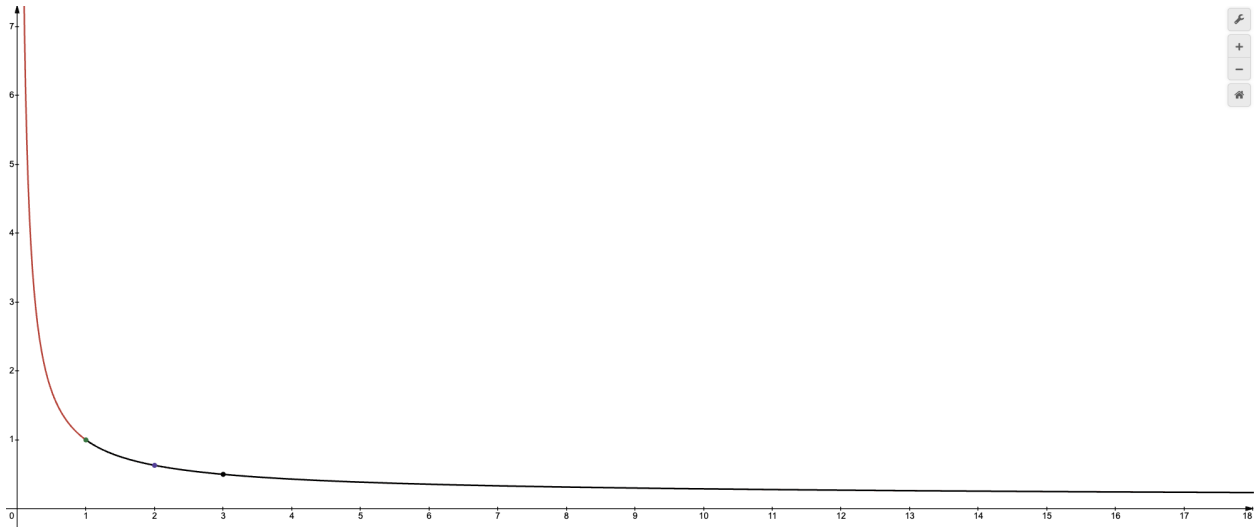


Figure 1: Top 3 ranks visualized

2 Why we should trust binary more

2.1 The task asks for one item, not a similar one

The evaluation is built around one specific item the user actually bought next, not a category. So the metric should ask the same question the task asks: did we name that item? The old metric asks a different question, how close in embedding space were our guesses, which is not the question the task is measuring. A user does not partially buy a similar guitar strap.

2.2 Closeness earns almost as much credit as being right

Because rel_i is a dot product, a wrong-but-similar item, say, a different guitar strap from the same brand, can score 0.9, close to what an exact match scores. A near-miss does not turn into a sale, so scoring it almost as high as a hit can make two systems that behave very differently look similar, when only one of them would actually convert.

2.3 The old metric can score high on a bad list

IDCG is computed from the same 3 similarity numbers the system just produced, sorted. If the system returns 3 items with nothing to do with t , but orders them by descending similarity to each other, NDCG still comes out close to 1. The metric is checking whether the list is sorted consistently with itself, not whether the list is any good. Binary NDCG does not have this property: its target is fixed at 1 (the true item at rank 1) regardless of what the system returned, so a low score always means the true item was missing from the top 3.

2.4 The same model does the recommending and the scoring

The recommender finds candidates by embedding similarity, using the same embedding model (`bge-m3`) that the old metric then uses to score those candidates. Any system built this way gets a similarity boost from the metric simply because it was built to optimize for similarity, independent of whether it ever finds t . Binary NDCG only checks item-id equality, so it does not depend on

which embedding model, if any, produced the recommendations, which also keeps it valid if a different kind of system is compared later.

2.5 It agrees with the metric we already report

We already report `top_one_match`: was t exactly at rank 1, yes or no. Binary NDCG@3 extends that same yes/no question to the top 3, with credit reduced for rank 2 or 3. Pairing it with the old similarity-based NDCG puts two different questions in the same report, one answers “did we get it right”, the other answers “were we in the neighborhood”, which makes them hard to read together. Pairing `top_one_match` with binary NDCG@3 keeps both numbers answering the same question.

2.6 The number has a direct meaning

A binary NDCG@3 of 0.4 means: on average, weighted by how early we found it, the exact next purchase lands in the top 3 about 40% of the time. A similarity-based NDCG@3 of 0.7 does not translate directly, it reflects how consistently the list was sorted by similarity, which is not the same as the list being useful.

3 What we’re not saying

We’re not saying to throw away the old similarity-based NDCG. It’s still useful as a side gauge for whether the ranking step is behaving reasonably. The point is narrower: for judging whether the system is actually good at its job, guessing the one item the user is about to buy, binary NDCG@3 should be the headline number, and the similarity version should be a secondary diagnostic, not the other way around.

4 Summary

	Similarity NDCG@3	Binary NDCG@3
Asks “did we get the exact item”	No	Yes
Can’t be fooled by a well-sorted bad list	No	Yes
Independent of the embedding model used to recommend	No	Yes
Same yes/no logic as <code>top_one_match</code>	No	Yes
Easy to explain in one sentence	No	Yes